

AWS Auto Scaling & CI/CD for a 3-Tier Application

Assignment Documentation Report
Highly Available, Self-Healing & Auto Scaling Backend on AWS

AWS Account ID	4485-1398-9308
Region	US East (N. Virginia) — us-east-1
Launch Template	backend-lt-20260827154757598800000001
Auto Scaling Group	backend-asg
Load Balancer	backend-alb
Target Group	backend-tg

Prepared for: Auto Scaling & CI/CD Assignment Submission

1. Architecture Diagram

The diagram below shows the 3-tier VPC design used for this assignment: a public subnet hosting the Frontend/Backend EC2 instances (managed by an Auto Scaling Group) behind an Application Load Balancer, and a private subnet hosting the Database, isolated from direct internet access. Outbound internet access for the public subnet is routed through a Route Table to an Internet Gateway (IGW). Users reach the application through the ALB, which forwards traffic to the Target Group registered with the Auto Scaling Group's instances. SSH access and CI/CD deployment are enabled on the backend instance for application updates.

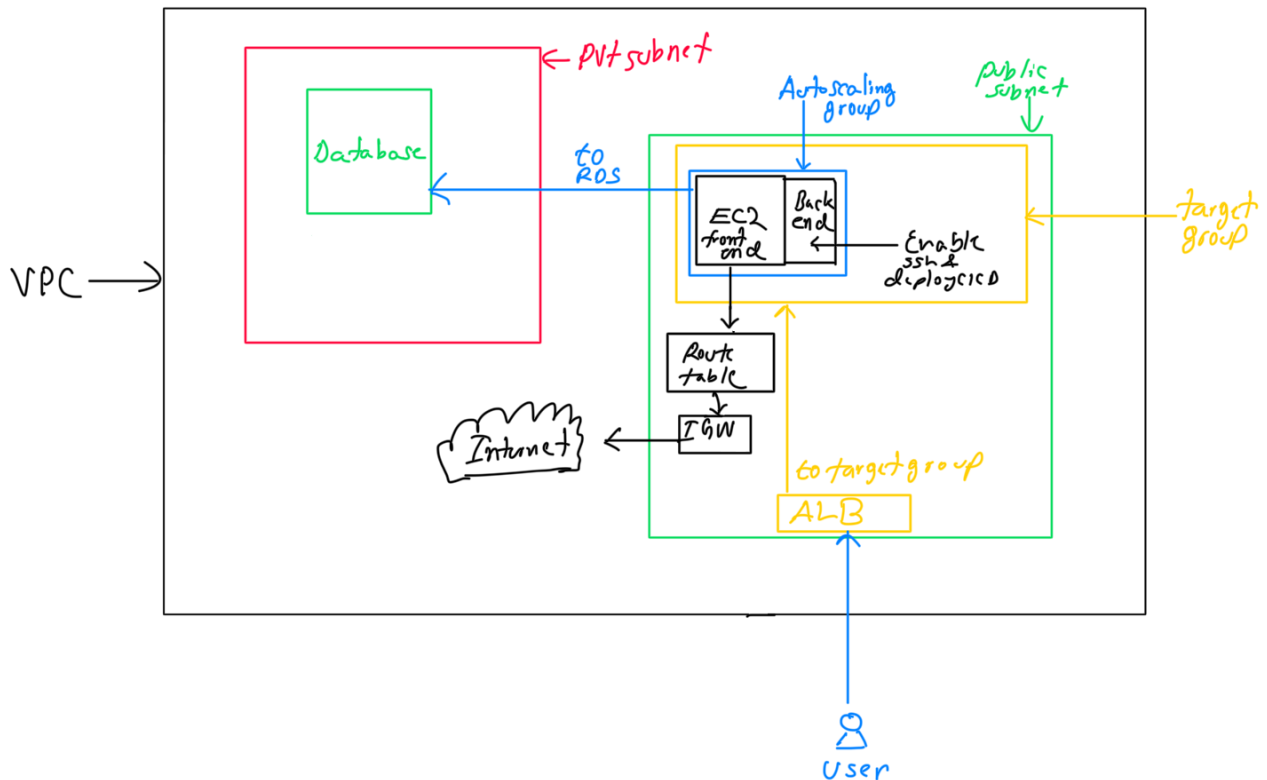


Figure 1 — 3-tier VPC architecture: public/private subnets, ALB, Target Group, Auto Scaling Group, EC2 (Frontend/Backend), Database, Route Table & Internet Gateway.

Key components:

- **VPC** — isolates the network; contains one public and one private subnet across multiple Availability Zones (us-east-1a, us-east-1b).
- **Public Subnet** — hosts the ALB and the Auto Scaling Group's EC2 instances (Frontend + Backend).
- **Private Subnet** — hosts the Database, reachable only from the backend tier.
- **ALB (Application Load Balancer)** — internet-facing; receives user traffic and forwards it to the Target Group.
- **Target Group (backend-tg)** — routes HTTP:80 traffic to healthy registered EC2 instances.

- **Auto Scaling Group (backend-asg)** — launches/terminates EC2 instances automatically based on health and scaling policies.
- **Route Table** → **Internet Gateway (IGW)** — provides outbound internet connectivity for the public subnet.

2. Launch Template Configuration

An EC2 Launch Template (**backend-lt-20260827154757598800000001**, ID: lt-02a0efd288f23024d) was created to standardize the configuration used by the Auto Scaling Group when launching new instances. It defines the AMI, instance type, key pair, and security group used for every auto-scaled backend instance.

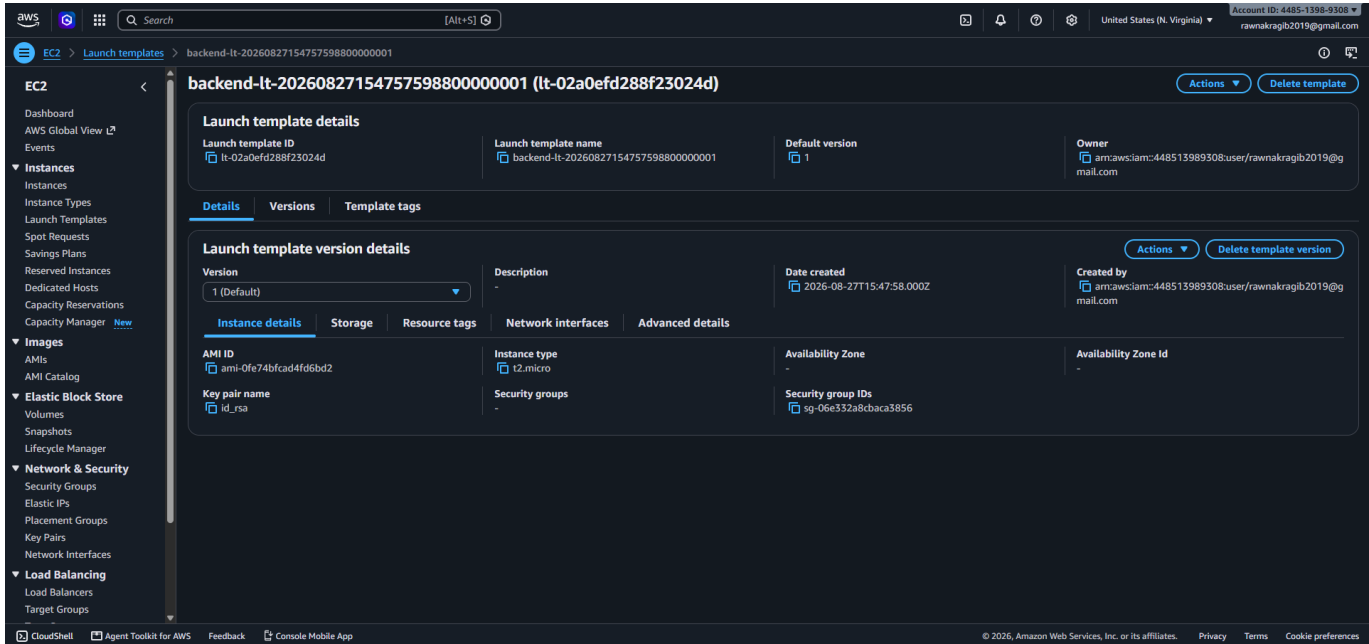


Figure 2 — Launch Template details: AMI ID (ami-0fe74bfcad4fd6bd2), instance type (t2.micro), key pair (id_rsa), and security group (sg-06e332a8cbaca3856).

Configuration summary:

- Launch Template ID: **lt-02a0efd288f23024d**, Default version: 1
- AMI ID: **ami-0fe74bfcad4fd6bd2**
- Instance type: **t2.micro**
- Key pair: **id_rsa**
- Security group: **sg-06e332a8cbaca3856**

3. Target Group & Application Load Balancer Configuration

A Target Group (**backend-tg**) was created with protocol HTTP on port 80, using instance target type. It is attached to the Application Load Balancer **backend-alb**, which listens on HTTP:80 and forwards all matching requests to the target group.

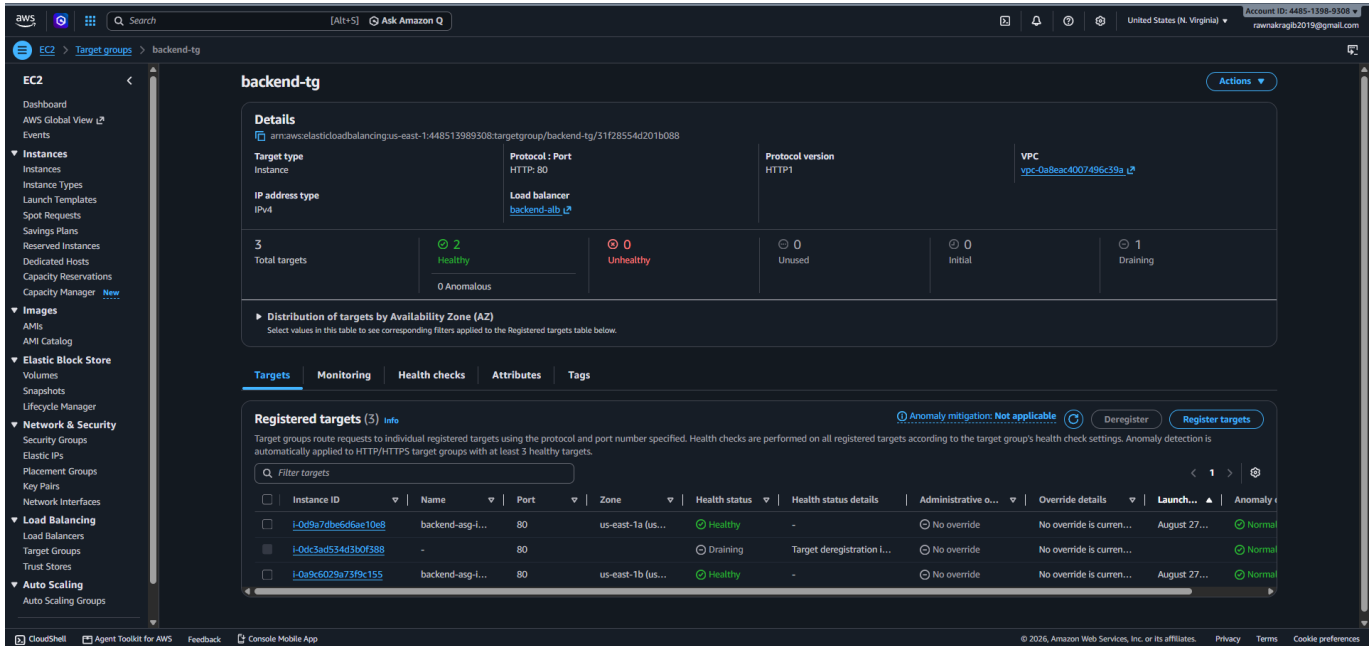


Figure 3 — Target Group (backend-tg) showing registered targets, health status, and the associated ALB (backend-alb).

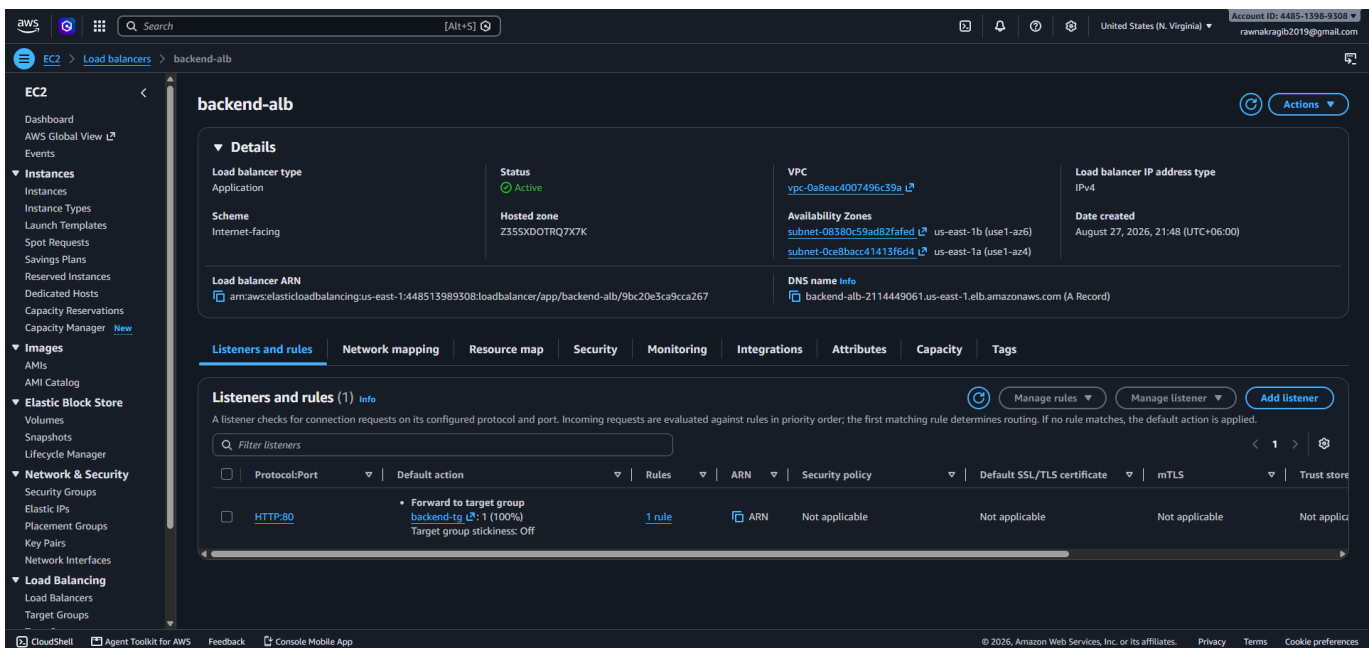


Figure 4 — Load Balancer (backend-alb) details: internet-facing, Application type, spanning two Availability Zones (us-east-1a and us-east-1b), with a listener on HTTP:80 forwarding to backend-tg.

Configuration summary:

- Target Group ARN:

arn:aws:elasticloadbalancing:us-east-1:448513989308:targetgroup/backend-tg/31f28554d201b088

- Protocol / Port: HTTP : 80, Protocol version: HTTP1
- Load Balancer type: Application, Scheme: Internet-facing, IP type: IPv4
- Availability Zones: us-east-1a and us-east-1b (multi-AZ for high availability)
- Listener: HTTP:80 → Forward to backend-tg (100%)

4. Auto Scaling Group Configuration

The Auto Scaling Group **backend-asg** uses the launch template above and is configured across two subnets (us-east-1a and us-east-1b) with balanced-best-effort Availability Zone distribution, ensuring the backend remains available even if one AZ has an issue.

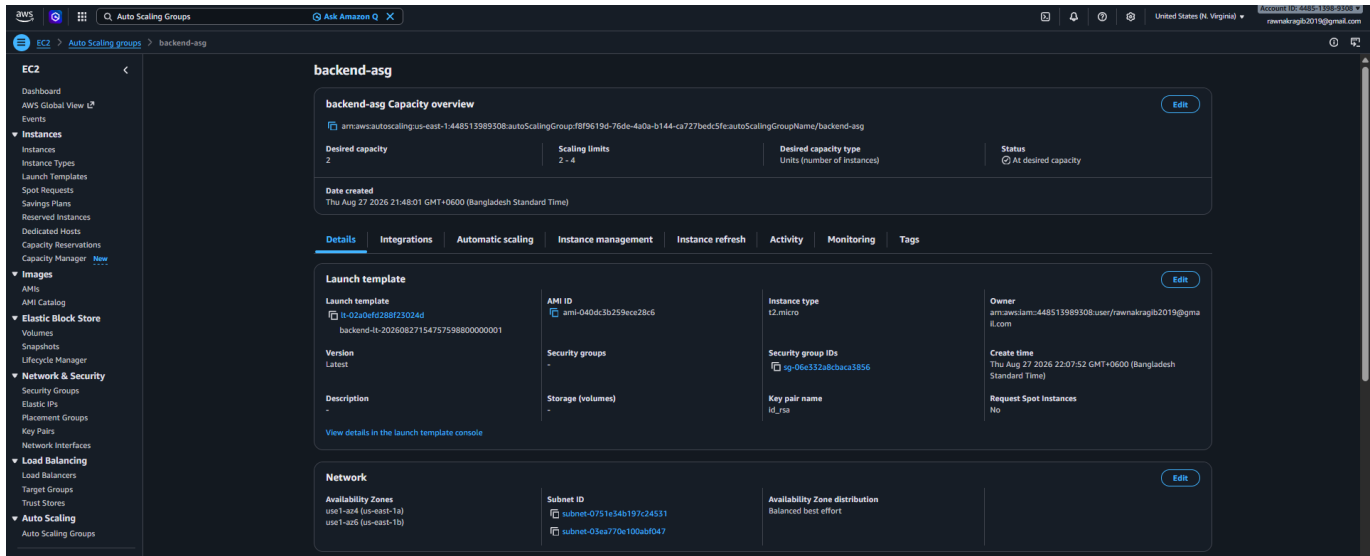


Figure 5 — backend-asg capacity overview: Desired capacity 2, Scaling limits 2–4, status "At desired capacity"; launch template, AMI, instance type, security group and multi-AZ subnet configuration.

Configuration summary:

- Auto Scaling Group name: **backend-asg**
- Minimum capacity: 2, Desired capacity: 2, Maximum capacity: 4
- Availability Zones: us-east-1a, us-east-1b (balanced best effort distribution)
- Launch template: backend-lt-20260827154757598800000001 (version: Latest)
- Instance type: t2.micro, Security group: sg-06e332a8cbaca3856

5. Health Check Configuration

A dedicated backend endpoint, `/health`, was implemented and returns a simple confirmation message ("Backend is Healthy") used by the ALB Target Group to determine instance health. The Target Group health check is configured with a 15-second interval, 5-second timeout, and requires 2 consecutive successes/failures to change state.

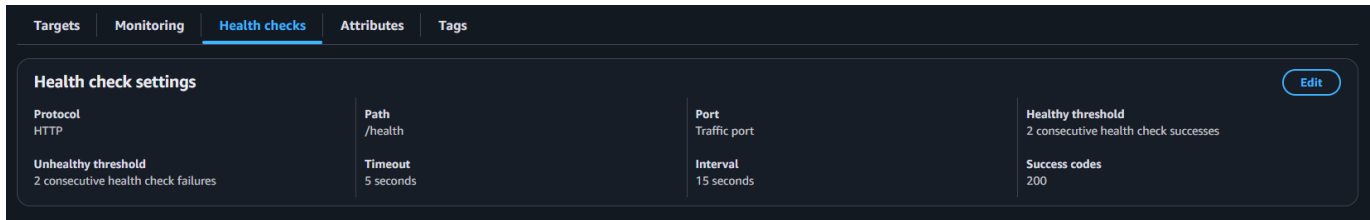


Figure 6 — Target Group health check settings: Protocol HTTP, Path `/health`, Port (traffic port), Healthy threshold 2, Unhealthy threshold 2, Timeout 5s, Interval 15s, Success code 200.

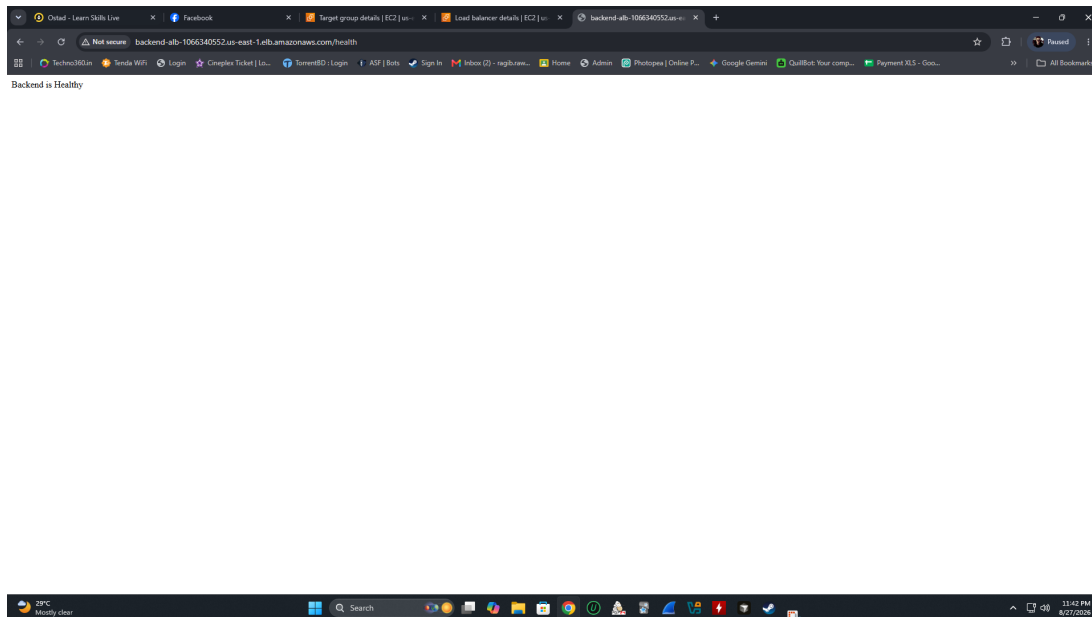


Figure 7 — Browser verification of the `/health` endpoint on the ALB DNS name, returning "Backend is Healthy".

Configuration summary:

- Health check protocol / path: HTTP on `/health`
- Success codes: 200
- Healthy threshold: 2 consecutive successes | Unhealthy threshold: 2 consecutive failures
- Timeout: 5 seconds | Interval: 15 seconds
- Auto Scaling Group health checks rely on the same ELB health check, so instances failing `/health` are automatically flagged unhealthy and replaced.

6. Scaling Policy Configuration

A Target Tracking scaling policy named **backend-cpu-target-tracking** was configured to maintain average CPU utilization at 50%. When CPU exceeds this target, the Auto Scaling Group adds capacity; when CPU drops, it removes capacity. New instances have a 300-second warm-up period before being included in the metric, and scale-in is enabled.

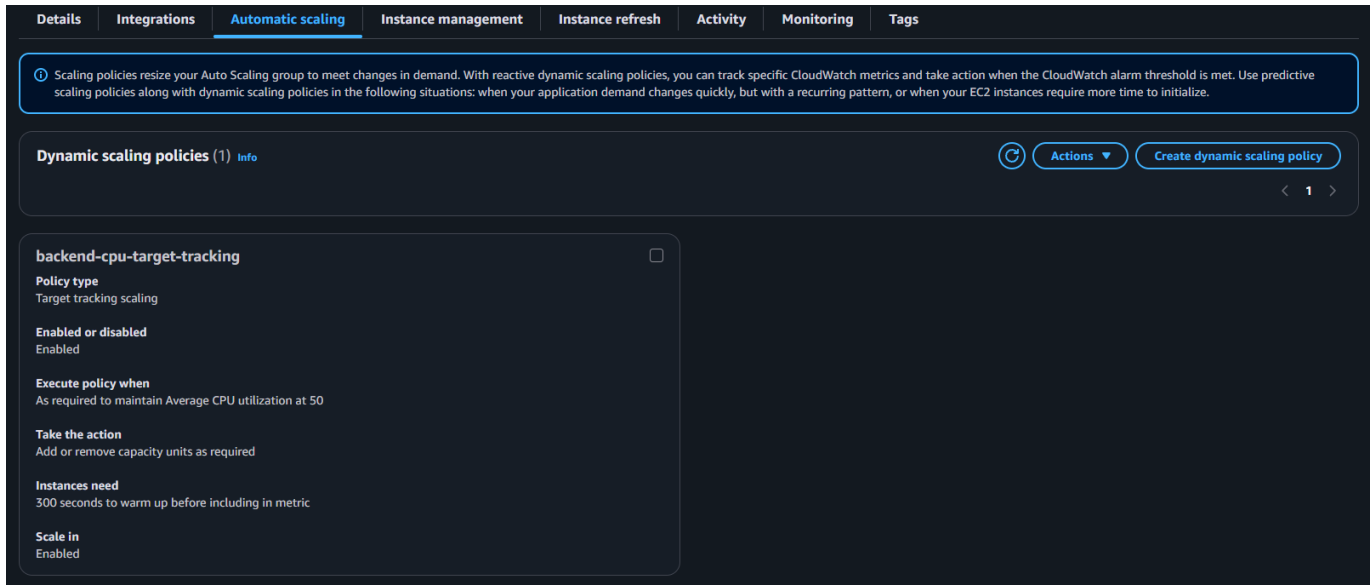


Figure 8 — Dynamic scaling policy "backend-cpu-target-tracking": Target tracking scaling, maintaining Average CPU Utilization at 50%, 300s instance warm-up, scale-in enabled.

Configuration summary:

- Policy type: Target Tracking Scaling
- Metric: Average CPU Utilization, Target value: 50%
- Instance warm-up: 300 seconds before inclusion in the metric
- Scale-in: Enabled
- Backed by a CloudWatch alarm (TargetTracking-backend-asg-AlarmHigh) that triggers scale-out when CPUUtilization > 50% for 3 datapoints within 3 minutes.

7. Self-Healing Test Evidence

To validate self-healing, a running backend instance was manually terminated/stopped. The ALB health check detected the instance as unhealthy, and the Auto Scaling Group's activity history confirms it automatically terminated the failed instance and launched a brand-new replacement to restore the desired capacity — with no manual intervention.

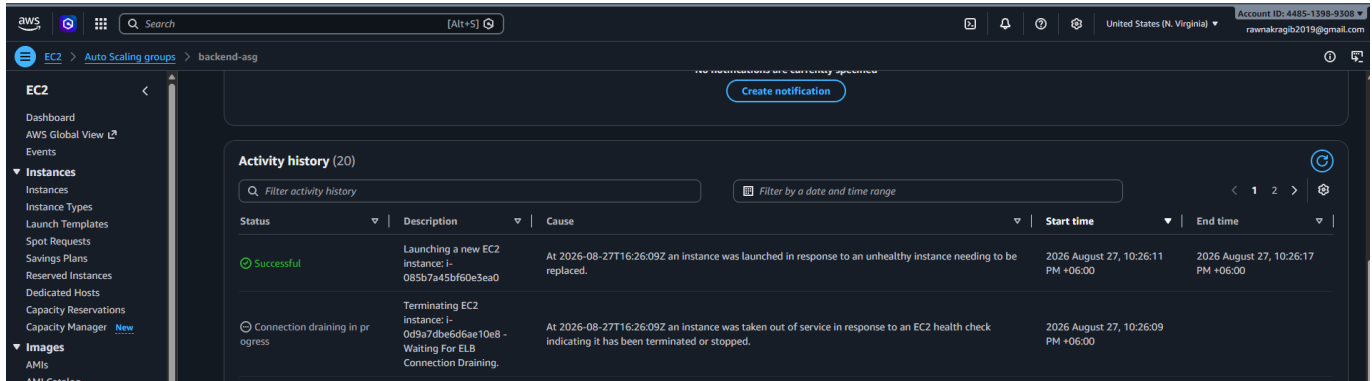


Figure 9 — ASG Activity History: instance `i-0d9a7dbe6d6ae10e8` taken out of service after failing its EC2 health check, and a new replacement instance (`i-085b7a45bf60e3ea0`) automatically launched moments later.

Observation:

- Unhealthy instance `i-0d9a7dbe6d6ae10e8` was detected via EC2/ELB health check as terminated/stopped and taken out of service (connection draining initiated).
- The Auto Scaling Group immediately launched a replacement instance `i-085b7a45bf60e3ea0` to bring the group back to its desired capacity — confirming self-healing works as expected.

Target Group view before and after replacement (draining old target, healthy new targets):

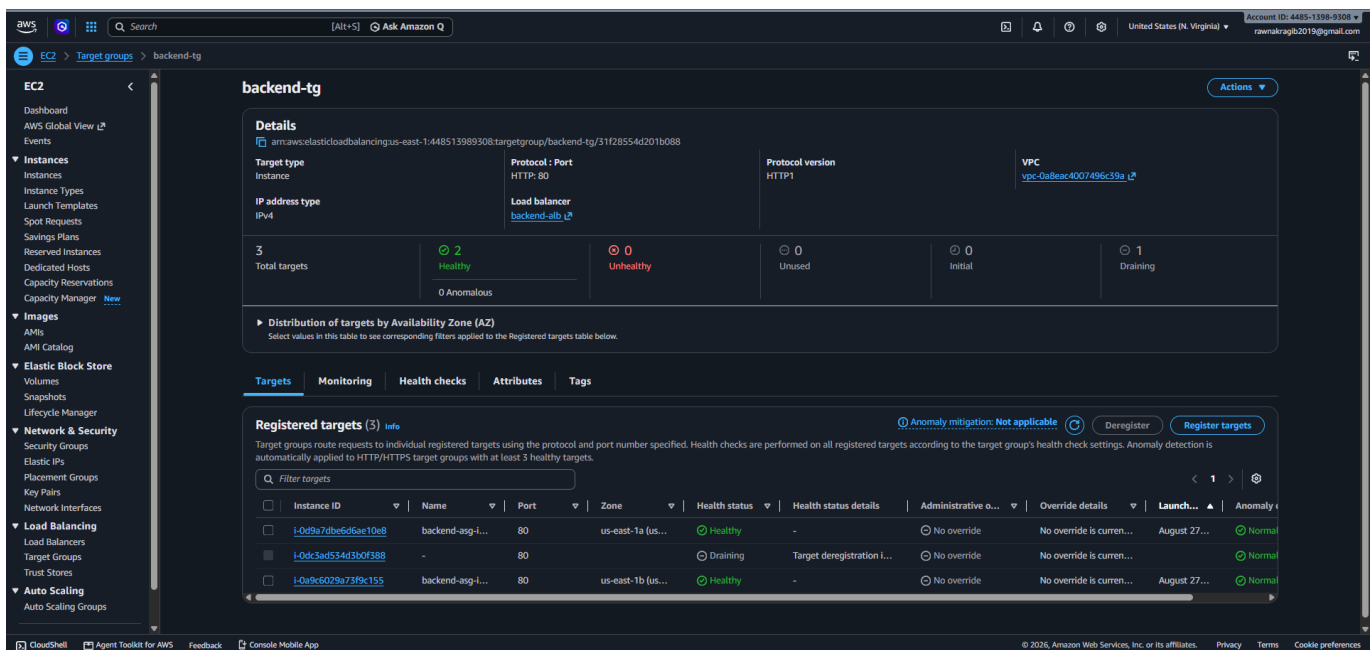


Figure 10 — Target group showing one instance in "Draining" state while a replacement is brought into service (2 healthy of 3 total).

8. Scale-Out / Scale-In Test Evidence

Load was generated against the backend to push average CPU utilization above the 50% target tracking threshold. This triggered the CloudWatch alarm and scale-out action; once the load was stopped, CPU utilization dropped back below the threshold and the group scaled back in.

8.1 Baseline — Alarm OK, Target Group healthy at desired capacity

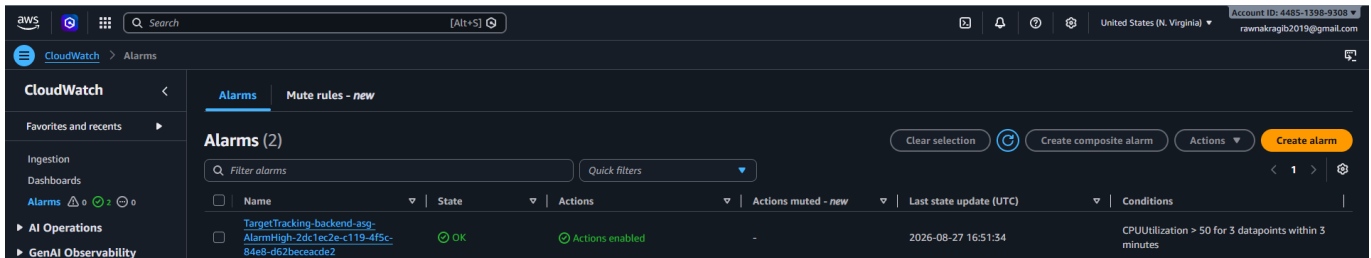


Figure 11 — CloudWatch alarm "TargetTracking-backend-asg-AlarmHigh" in OK state before load was applied.

8.2 Load Applied — CPU spikes above 50%, alarm triggers scale-out

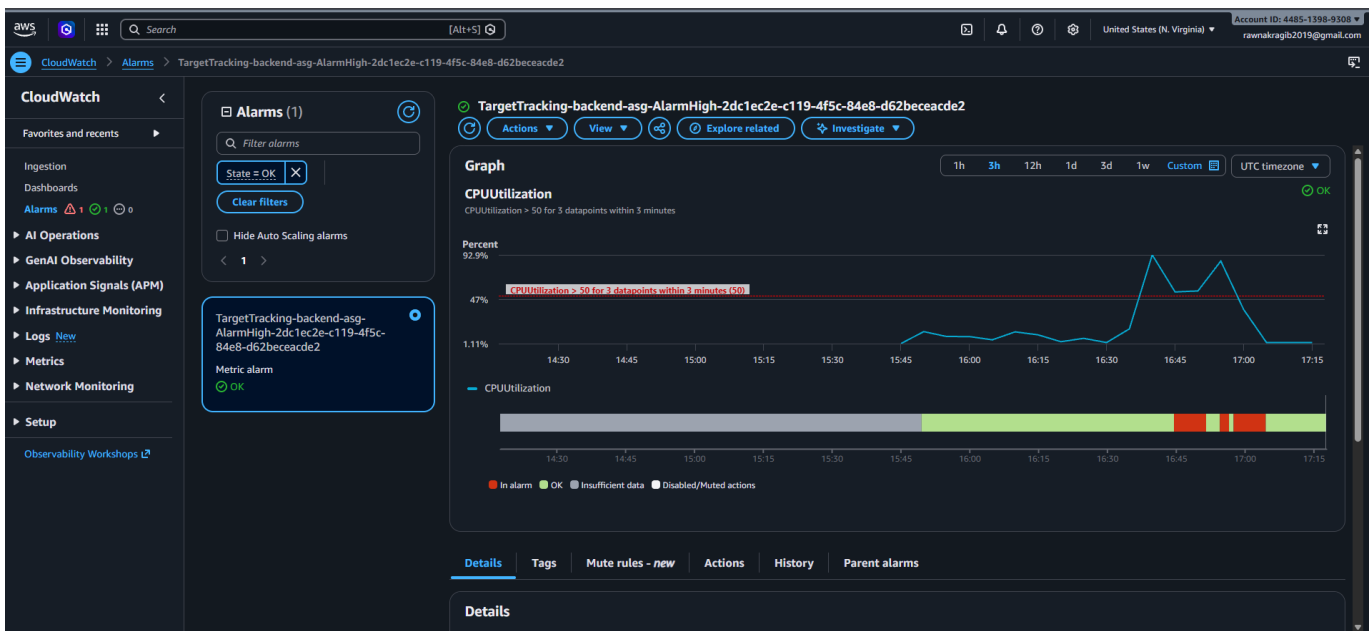


Figure 12 — CPUUtilization graph spiking to ~93% (well above the 50% threshold), causing the alarm to move to "In alarm" and back to "OK" as the ASG reacted.

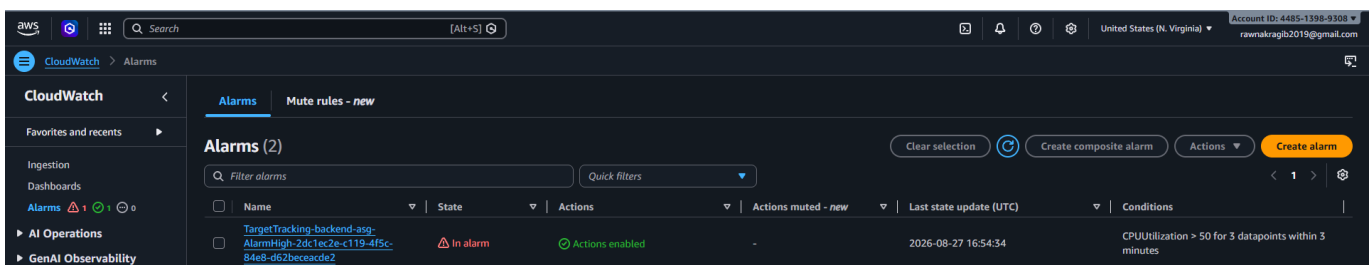
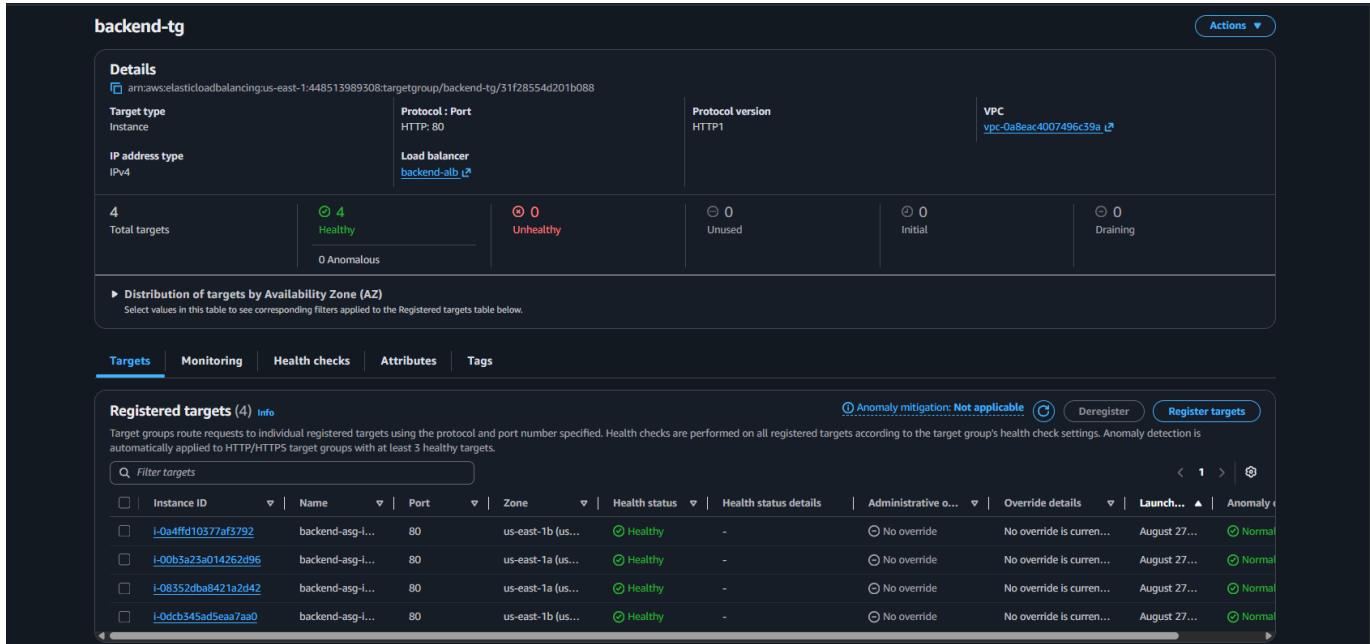


Figure 13 — CloudWatch alarm state changed to "In alarm" as CPUUtilization exceeded 50% for 3 datapoints within 3 minutes.

8.3 Scale-Out Confirmed — additional healthy targets registered



backend-tg

Details

arn:aws:elasticloadbalancing:us-east-1:448513989308:targetgroup/backend-tg/31f28554d201b088

Target type: Instance

Protocol: Port: HTTP: 80

Protocol version: HTTP1

VPC: vpc-0a8eac4007496c39a

IP address type: IPv4

Load balancer: backend-alb

4 Total targets

2 Healthy

0 Unhealthy

0 Unused

0 Initial

0 Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

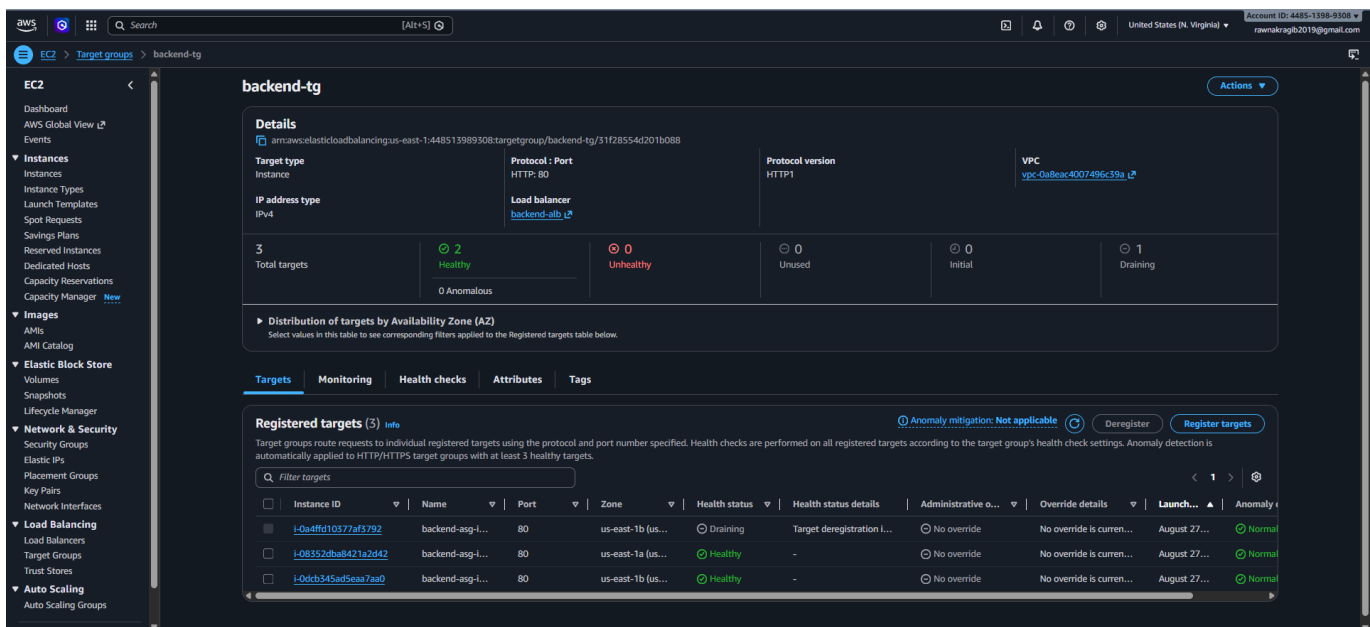
Registered targets (4)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anomaly
i-0a4ffdf10377af3792	backend-asg-l...	80	us-east-1b (us...	Healthy	-	No override	No override is curren...	August 27...	Normal
i-00b3a23a014262d96	backend-asg-l...	80	us-east-1a (us...	Healthy	-	No override	No override is curren...	August 27...	Normal
i-08352dba8421a2d42	backend-asg-l...	80	us-east-1a (us...	Healthy	-	No override	No override is curren...	August 27...	Normal
i-0dcb345ad5eaa7aa0	backend-asg-l...	80	us-east-1b (us...	Healthy	-	No override	No override is curren...	August 27...	Normal

Figure 14 — Target Group scaled out to 4 healthy registered targets (up from the desired baseline of 2), confirming the Auto Scaling Group added capacity in response to the CPU alarm.

8.4 Scale-In — after load stopped, capacity was reduced back down



backend-tg

Details

arn:aws:elasticloadbalancing:us-east-1:448513989308:targetgroup/backend-tg/31f28554d201b088

Target type: Instance

Protocol: Port: HTTP: 80

Protocol version: HTTP1

VPC: vpc-0a8eac4007496c39a

IP address type: IPv4

Load balancer: backend-alb

3 Total targets

2 Healthy

0 Unhealthy

0 Unused

0 Initial

1 Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (3)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anomaly
i-0a4ffdf10377af3792	backend-asg-l...	80	us-east-1b (us...	Draining	Target deregistration l...	No override	No override is curren...	August 27...	Normal
i-08352dba8421a2d42	backend-asg-l...	80	us-east-1a (us...	Healthy	-	No override	No override is curren...	August 27...	Normal
i-0dcb345ad5eaa7aa0	backend-asg-l...	80	us-east-1b (us...	Healthy	-	No override	No override is curren...	August 27...	Normal

Figure 15 — Target Group returning toward baseline capacity: one target draining while the group scales back in as CPU utilization normalized after the load was removed.

Summary of results:

- Under load, CPUUtilization rose to ~93%, well above the 50% target — the target tracking policy correctly triggered a scale-out, increasing registered healthy targets from 2 to 4.

- After the load was stopped, CPU utilization fell back under the 50% threshold, and the Auto Scaling Group began draining and terminating the extra instances to scale back toward the desired capacity of 2.
- This confirms the target tracking scaling policy responds correctly to both increased and decreased demand.

9. Application Endpoint Verification

The application was accessed directly through the ALB's public DNS name to confirm end-to-end connectivity from the internet, through the ALB and Target Group, to the backend EC2 instances running on Ubuntu.

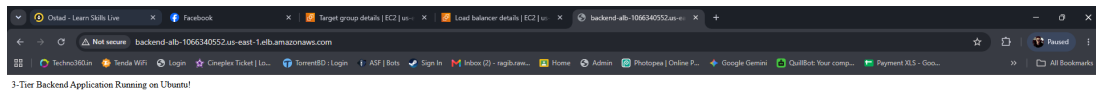


Figure 16 — Backend accessed via the ALB DNS name (*backend-alb-1066340552.us-east-1.elb.amazonaws.com*), confirming: "3-Tier Backend Application Running on Ubuntu!"

10. Short Documentation of the Complete Setup

This section summarizes the end-to-end setup implemented for this assignment, converting a single-instance backend into a highly available, self-healing, auto-scaling architecture on AWS.

Architecture

A VPC was designed with a public subnet (ALB + Auto Scaling Group instances) and a private subnet (Database), spread across two Availability Zones (us-east-1a and us-east-1b) for redundancy. Outbound internet access flows through a Route Table to an Internet Gateway.

Compute & Scaling Layer

- **Launch Template** (backend-lt-...): standardizes AMI, instance type (t2.micro), key pair, and security group for every new instance.
- **Target Group** (backend-tg): HTTP:80, instance target type, health-checked via /health.
- **Application Load Balancer** (backend-alb): internet-facing, multi-AZ, listens on HTTP:80 and forwards to backend-tg.
- **Auto Scaling Group** (backend-asg): min 2 / desired 2 / max 4 instances, spans both AZs, uses the launch template above.

Health Checks & Self-Healing

The backend exposes a /health endpoint returning HTTP 200 with "Backend is Healthy". The Target Group polls this endpoint every 15 seconds (5s timeout, 2 consecutive pass/fail thresholds). Instances that fail health checks are automatically deregistered, drained, and terminated by the ASG, which immediately launches a replacement — verified in Section 7 by manually terminating an instance and observing an automatic replacement in the ASG activity history.

Scaling Policy

A Target Tracking policy (backend-cpu-target-tracking) keeps average CPU utilization at 50%, backed by a CloudWatch alarm (CPUUtilization > 50% for 3 datapoints within 3 minutes). Instances get a 300-second warm-up before being counted in the metric, and scale-in is enabled. Under generated load, CPU spiked to ~93%, the alarm fired, and the target group scaled out from 2 to 4 healthy targets; after load was removed, the group scaled back in toward its desired capacity.

Result

The backend is now fault-tolerant (multi-AZ, self-healing on instance failure) and elastic (automatically scales out under load and back in when load subsides), fulfilling the assignment's objective of converting a single-instance backend into a highly available, auto scaling, and self-healing architecture.